Apache con virtual hosts, red aislada y cloud-init en KVM/libvirt

Esta guía muestra cómo montar un escenario local con **QEMU/KVM + libvirt** para practicar Apache. Se crean dos máquinas Debian a partir de una imagen cloud y discos QCOW2 enlazados:

- `servidorweb`: Apache y dos sitios web.
- `cliente`: máquina desde la que se hacen las pruebas.

Cada máquina tendrá dos tarjetas de red:

Máquina	Red NAT `default`	Red aislada `red_sri_web`
`servidorweb`	DHCP, por ejemplo `192.168.122.63`	`10.0.0.1/24`
`cliente`	DHCP, por ejemplo `192.168.122.216`	`10.0.0.2/24`

La red NAT permite instalar paquetes y conectarse por SSH desde el anfitrión. La red aislada se usa para que `cliente` acceda a `servidorweb` mediante `10.0.0.1`.

> **Importante:** los nombres de interfaces usados en esta guía son `ens3` para la primera NIC y `ens4` para la segunda. Se crean en ese orden con `virt-install`.

Requisitos

En el anfitrión Debian se necesitan QEMU/KVM, libvirt, `virt-install`, cloud-init tools y una imagen cloud de Debian. Esta guía parte de una imagen base ya descargada en:

```
/var/lib/libvirt/images/debian-13-generic-amd64.qcow2
```

También se asume una clave pública SSH disponible en:

```
~/.ssh/id_rsa.pub
```

Si aún faltan las herramientas principales:

```
sudo apt update
sudo apt install -y qemu-kvm libvirt-daemon-system libvirt-clients virtinst
cloud-image-utils
```

Comprueba que trabajas sobre la conexión de sistema:

```
sudo virsh -c qemu:///system list --all
sudo virsh -c qemu:///system net-list --all
```

1. Crear la red aislada

La red `red_sri_web` no tiene NAT, gateway, DHCP ni IP en el host. Solo conecta las interfaces que se conecten a ella.

```
cat > /tmp/red_sri_web.xml <<'EOF'
<network>
  <name>red_sri_web</name>
  <bridge name='virbr-sri' stp='on' delay='0'/>
</network>
EOF

sudo virsh -c qemu:///system net-define /tmp/red_sri_web.xml
sudo virsh -c qemu:///system net-autostart red_sri_web
sudo virsh -c qemu:///system net-start red_sri_web
sudo virsh -c qemu:///system net-list --all
```

La última orden debe mostrar una línea similar a:

```
red_sri_web        activo   si   si
```

La red NAT `default` no se modifica. Ya proporciona DHCP y salida a Internet para las máquinas virtuales.

2. Crear discos QCOW2 enlazados

Los discos enlazados usan la imagen cloud como backing file. Las VMs ven discos de 12 GB, pero los discos nuevos empiezan ocupando solo unos KB y guardan únicamente los cambios.

```
sudo qemu-img create -f qcow2 \
  -F qcow2 \
  -b /var/lib/libvirt/images/debian-13-generic-amd64.qcow2 \
  /var/lib/libvirt/images/sri-servidorweb.qcow2 12G

sudo qemu-img create -f qcow2 \
  -F qcow2 \
  -b /var/lib/libvirt/images/debian-13-generic-amd64.qcow2 \
  /var/lib/libvirt/images/sri-cliente.qcow2 12G

sudo chown libvirt-qemu:libvirt-qemu \
  /var/lib/libvirt/images/sri-servidorweb.qcow2 \
  /var/lib/libvirt/images/sri-cliente.qcow2
```

Para comprobar que el primer disco tiene como base la imagen Debian:

```
sudo qemu-img info --backing-chain /var/lib/libvirt/images/sri-servidorweb.qcow2
```

La salida debe incluir algo parecido a:

```
image: /var/lib/libvirt/images/sri-servidorweb.qcow2
virtual size: 12 GiB
backing file: /var/lib/libvirt/images/debian-13-generic-amd64.qcow2
```

3. Cloud-init de `servidorweb`

Se crean tres ficheros: `user-data`, `meta-data` y `network-config`. La configuración crea el usuario `sergio`, añade la clave pública, permite sudo sin contraseña, establece el hostname y configura las dos interfaces.

```
mkdir -p "$HOME/sri-cloud-init"

cat > "$HOME/sri-cloud-init/user-data-servidorweb.yaml" <<EOF
#cloud-config
hostname: servidorweb
manage_etc_hosts: true

users:
  - name: sergio
    groups: [sudo]
    shell: /bin/bash
    sudo: ALL=(ALL) NOPASSWD:ALL
    ssh_authorized_keys:
      - $(cat ~/.ssh/id_rsa.pub)

ssh_pwauth: false

package_update: true
packages:
  - apache2
  - curl
EOF

cat > "$HOME/sri-cloud-init/meta-data-servidorweb.yaml" <<'EOF'
instance-id: sri-servidorweb-001
local-hostname: servidorweb
EOF

cat > "$HOME/sri-cloud-init/network-config-servidorweb.yaml" <<'EOF'
version: 2
ethernets:
  ens3:
    dhcp4: true
  ens4:
    dhcp4: false
    addresses:
      - 10.0.0.1/24
EOF
```

Se genera la ISO NoCloud que leerá cloud-init. Se usa `$HOME` y no `~` después de `--network-config=`, porque en ese caso la shell no expande la tilde.

```
cloud-localds \
  --network-config="$HOME/sri-cloud-init/network-config-servidorweb.yaml" \
  "$HOME/sri-cloud-init/seed-servidorweb.iso" \
  "$HOME/sri-cloud-init/user-data-servidorweb.yaml" \
  "$HOME/sri-cloud-init/meta-data-servidorweb.yaml"
```

4. Crear `servidorweb`

La primera NIC se conecta a `default` y la segunda a `red_sri_web`. El orden es importante para que cloud-init aplique DHCP sobre `ens3` y la IP fija sobre `ens4`.

```
sudo virt-install \
  --connect qemu:///system \
  --name servidorweb \
  --memory 1024 \
  --vcpus 1 \
  --import \
  --osinfo detect=on,require=off \
  --disk path=/var/lib/libvirt/images/sri-
servidorweb.qcow2,format=qcow2,bus=virtio \
  --disk path="$HOME/sri-cloud-init/seed-servidorweb.iso",device=cdrom \
  --network network=default,model=virtio \
  --network network=red_sri_web,model=virtio \
  --graphics spice \
  --video virtio \
  --noautoconsole
```

El aviso de `virt-install` sobre usar el perfil `generic` no impide arrancar una imagen cloud.

Para descubrir la IP recibida por DHCP:

```
sudo virsh -c qemu:///system net-dhcp-leases default
```

En este ejemplo, `servidorweb` recibió `192.168.122.63`. Se puede entrar por SSH:

```
ssh -i ~/.ssh/id_rsa sergio@192.168.122.63
```

Dentro de la VM, comprueba las interfaces y rutas:

```
hostnamectl
ip -br a
ip route
```

El resultado esperado es equivalente a:

```
ens3  UP  192.168.122.63/24
ens4  UP  10.0.0.1/24

default via 192.168.122.1 dev ens3
10.0.0.0/24 dev ens4
```

Si Apache no estuviera instalado aún, instálalo y actívalo:

```
sudo apt update
sudo apt install -y apache2
sudo systemctl enable --now apache2
```

5. Cloud-init y creación de `cliente`

Desde el anfitrión, crea los ficheros de cloud-init del cliente. Su única diferencia importante es el hostname y la IP `10.0.0.2/24` de la red aislada.

```
cat > "$HOME/sri-cloud-init/user-data-cliente.yaml" <<EOF
#cloud-config
hostname: cliente
manage_etc_hosts: true

users:
  - name: sergio
    groups: [sudo]
    shell: /bin/bash
    sudo: ALL=(ALL) NOPASSWD:ALL
    ssh_authorized_keys:
      - $(cat ~/.ssh/id_rsa.pub)

ssh_pwauth: false

package_update: true
packages:
  - curl
EOF

cat > "$HOME/sri-cloud-init/meta-data-cliente.yaml" <<'EOF'
instance-id: sri-cliente-001
local-hostname: cliente
EOF

cat > "$HOME/sri-cloud-init/network-config-cliente.yaml" <<'EOF'
version: 2
ethernets:
  ens3:
    dhcp4: true
  ens4:
    dhcp4: false
    addresses:
      - 10.0.0.2/24
EOF

cloud-localds \
  --network-config="$HOME/sri-cloud-init/network-config-cliente.yaml" \
  "$HOME/sri-cloud-init/seed-cliente.iso" \
  "$HOME/sri-cloud-init/user-data-cliente.yaml" \
  "$HOME/sri-cloud-init/meta-data-cliente.yaml"
```

Crea la VM:

```
sudo virt-install \
  --connect qemu:///system \
  --name cliente \
  --memory 1024 \
  --vcpus 1 \
  --import \
  --osinfo detect=on,require=off \
  --disk path=/var/lib/libvirt/images/sri-cliente.qcow2,format=qcow2,bus=virtio
\
  --disk path="$HOME/sri-cloud-init/seed-cliente.iso",device=cdrom \
  --network network=default,model=virtio \
  --network network=red_sri_web,model=virtio \
  --graphics spice \
  --video virtio \
  --noautoconsole
```

Obtén su IP NAT y entra por SSH:

```
sudo virsh -c qemu:///system net-dhcp-leases default
ssh -i ~/.ssh/id_rsa sergio@192.168.122.216
```

Dentro del cliente, comprueba la red aislada y la conectividad hacia el servidor:

```
hostnamectl
ip -br a
ip route
ping -c 3 10.0.0.1
curl -I http://10.0.0.1
```

Se espera una configuración parecida a:

```
ens3  UP  192.168.122.216/24
ens4  UP  10.0.0.2/24

default via 192.168.122.1 dev ens3
10.0.0.0/24 dev ens4
```

El `ping` debe responder y `curl -I http://10.0.0.1` debe devolver `HTTP/1.1 200 OK` cuando Apache esté activo.

6. Crear el contenido de los sitios

En `servidorweb`, crea los directorios:

```
sudo mkdir -p \
  /var/www/sitio1/intranet \
  /var/www/sitio1/secreto \
  /var/www/sitio1/articulos \
  /var/www/sitio2
```

Crea la página principal del sitio 1:

```
sudo tee /var/www/sitio1/index.html >/dev/null <<'EOF'
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <title>www.sitio1.org</title>
</head>
<body>
  <h1>www.sitio1.org</h1>
  <p>Esta es la página principal del sitio 1.</p>
</body>
</html>
EOF
```

Crea la página principal del sitio 2:

```
sudo tee /var/www/sitio2/index.html >/dev/null <<'EOF'
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <title>www.sitio2.org</title>
</head>
<body>
  <h1>www.sitio2.org</h1>
  <p>Esta es la página principal del sitio 2.</p>
</body>
</html>
EOF
```

Crea el contenido de las rutas protegidas y del artículo:

```
sudo tee /var/www/sitio1/intranet/index.html >/dev/null <<'EOF'
<h1>Intranet de www.sitio1.org</h1>
<p>Solo accesible desde cliente (10.0.0.2).</p>
EOF

sudo tee /var/www/sitio1/secreto/index.html >/dev/null <<'EOF'
<h1>Zona secreta de www.sitio1.org</h1>
<p>Acceso permitido desde cliente o con autenticación básica.</p>
EOF

sudo tee /var/www/sitio1/articulos/bienvenida.html >/dev/null <<'EOF'
<h1>Artículo de bienvenida</h1>
<p>Contenido servido mediante reescritura interna.</p>
EOF
```

7. Configurar virtual hosts en Apache

El fichero `001-sitio2.conf` se crea primero para que `www.sitio2.org` sea el virtual host por defecto cuando se acceda mediante la IP.

```
sudo tee /etc/apache2/sites-available/001-sitio2.conf >/dev/null <<'EOF'
<VirtualHost *:80>
    ServerName www.sitio2.org
    ServerAlias sitio2.org
    DocumentRoot /var/www/sitio2

    <Directory /var/www/sitio2>
        Options -Indexes +FollowSymLinks
        AllowOverride None
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/sitio2_error.log
    CustomLog ${APACHE_LOG_DIR}/sitio2_access.log combined
</VirtualHost>
EOF
```

Crea el virtual host principal de `www.sitio1.org`:

```
sudo tee /etc/apache2/sites-available/002-sitio1.conf >/dev/null <<'EOF'
<VirtualHost *:80>
    ServerName www.sitio1.org
    ServerAlias sitio1.org
    DocumentRoot /var/www/sitio1

    <Directory /var/www/sitio1>
        Options -Indexes +FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    <Location /intranet>
        Require ip 10.0.0.2
    </Location>

    <Location /secreto>
        AuthType Basic
        AuthName "Zona secreta"
        AuthUserFile /etc/apache2/.htpasswd

        <RequireAny>
            Require ip 10.0.0.2
            Require valid-user
        </RequireAny>
    </Location>

    ErrorLog ${APACHE_LOG_DIR}/sitio1_error.log
    CustomLog ${APACHE_LOG_DIR}/sitio1_access.log combined
</VirtualHost>
EOF
```

Activa los sitios, activa `mod_rewrite`, valida la sintaxis y recarga Apache:

```
sudo a2dissite 000-default.conf
sudo a2ensite 001-sitio2.conf 002-sitio1.conf
sudo a2enmod rewrite
sudo apachectl configtest
sudo systemctl reload apache2
```

Opcionalmente, se puede eliminar el aviso de `ServerName` global:

```
echo 'ServerName servidorweb' | sudo tee /etc/apache2/conf-
available/servername.conf
sudo a2enconf servername
sudo systemctl reload apache2
sudo apachectl configtest
```

8. Configurar autenticación básica

Crea un usuario para la ruta `/secreto`. El comando pide la contraseña de forma interactiva:

```
sudo htpasswd -c /etc/apache2/.htpasswd alumno
```

Si necesitas cambiar la contraseña sin borrar el fichero, no uses `-c`:

```
sudo htpasswd /etc/apache2/.htpasswd alumno
```

9. Reescritura con `.htaccess`

La directiva `AllowOverride All` del virtual host de sitio 1 permite usar `.htaccess`. Crea la regla:

```
sudo tee /var/www/sitio1/.htaccess >/dev/null <<'EOF'
RewriteEngine On
RewriteRule ^blog/(.+)$ articulos/$1.html [L]
EOF

sudo apachectl configtest
sudo systemctl reload apache2
```

La URL `/blog/bienvenida` se reescribe internamente a `/articulos/bienvenida.html`.

10. Resolución estática de nombres

En `cliente`, los dominios deben apuntar a la IP interna del servidor para que el tráfico pase por la red aislada:

```
echo '10.0.0.1 www.sitio1.org www.sitio2.org' | sudo tee -a /etc/hosts
```

En el anfitrión, los dominios se asocian a la IP NAT de `servidorweb`, porque el anfitrión no forma parte de la red aislada:

```
echo '192.168.122.63 www.sitio1.org www.sitio2.org' | sudo tee -a /etc/hosts
```

Comprueba la resolución desde el cliente:

```
getent hosts www.sitio1.org
getent hosts www.sitio2.org
```

11. Comprobaciones finales

Virtual hosts y acceso por IP

Desde `cliente`:

```
curl http://www.sitio1.org
curl http://www.sitio2.org
curl -I http://10.0.0.1
curl http://10.0.0.1
```

Los dos primeros comandos deben mostrar las páginas diferentes de cada sitio. El acceso por IP debe mostrar el contenido de `www.sitio2.org`.

La razón es que la petición con IP no coincide con ningún `ServerName`. Apache usa entonces el primer virtual host habilitado en el puerto 80, que es `001-sitio2.conf`.

Puedes comprobarlo con:

```
sudo apachectl -S
```

Debe aparecer `www.sitio2.org` como `default server`.

Acceso a `/intranet`

Desde `cliente`:

```
curl -I http://www.sitio1.org/intranet/
```

La respuesta esperada es `HTTP/1.1 200 OK` porque el origen es `10.0.0.2`.

Desde el anfitrión:

```
curl -I http://www.sitio1.org/intranet/
```

La respuesta esperada es `HTTP/1.1 403 Forbidden`, porque el anfitrión no llega desde `10.0.0.2`.

Acceso a `/secreto`

Desde `cliente`:

```
curl -I http://www.sitio1.org/secreto/
```

La respuesta debe ser `HTTP/1.1 200 OK` sin autenticación.

Desde el anfitrión, sin credenciales:

```
curl -I http://www.sitio1.org/secreto/
```

La respuesta debe ser `HTTP/1.1 401 Unauthorized` y mostrar la cabecera `WWW-Authenticate`.

Desde el anfitrión, con credenciales:

```
curl -I -u alumno:clave123 http://www.sitio1.org/secreto/
```

La respuesta debe ser `HTTP/1.1 200 OK`.

Reescritura

Desde `cliente` o desde el anfitrión:

```
curl -I http://www.sitio1.org/blog/bienvenida
```

La respuesta debe ser `HTTP/1.1 200 OK` y no debe incluir una cabecera `Location`.

Una **redirección** devuelve normalmente un código `301` o `302`, incluye `Location` y el navegador hace otra petición a una nueva URL. Una **reescritura** cambia internamente el fichero servido: el cliente mantiene `/blog/bienvenida` como URL, pero Apache sirve el contenido de `/articulos/bienvenida.html`.

Problemas habituales

`cloud-localds` no encuentra el fichero de red

Esto puede fallar:

```
cloud-localds --network-config=~/sri-cloud-init/network-config.yaml ...
```

La tilde no se expande al quedar después del signo igual. Usa `$HOME`:

```
cloud-localds --network-config="$HOME/sri-cloud-init/network-config.yaml" ...
```

Apache devuelve 301 en `/intranet` o `/secreto`

Las rutas son directorios. Si se llama sin barra final, Apache puede responder con una redirección hacia la misma URL terminada en `/`.

Usa:

```
curl -I http://www.sitio1.org/intranet/
curl -I http://www.sitio1.org/secreto/
```

La autenticación devuelve 401 incluso usando `-u`

La contraseña indicada en `curl` debe ser exactamente la que se escribió con `htpasswd`. Se puede cambiar con:

```
sudo htpasswd /etc/apache2/.htpasswd alumno
```

Después repite la prueba con la nueva contraseña.